ENSAM Casablanca

Filière: Intelligence Artificielle et Génie Informatique (IAGI 2)

Multi-Agent Deep Reinforcement Learning for Cooperative Soccer:

From Q-Learning to Proximal Policy Optimisation in MuJoCo

IMAD EDDINE OUKRATI & ECHCHYOUGHI MOHAMED

Supervisor: Prof. Hirchoua Badr

A report submitted in partial fulfilment of the requirements of
ENSAM Casablanca
Module: Deep Reinforcement Learning

May 28, 2026

Declaration

I, IMAD EDDINE OUKRATI and ECHCHYOUGHI MOHAMED, of the Intelligence Artificielle et Génie Informatique (IAGI 2) programme, ENSAM Casablanca, confirm that this is our own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

IMAD EDDINE OUKRATI & ECHCHYOUGHI MOHAMED
May 28, 2026

Abstract

This report presents a multi-agent Deep Reinforcement Learning (DRL) system applied to a cooperative and competitive 2-versus-2 football task simulated in the MuJoCo physics engine. The investigated problem is how autonomous agents can learn to coordinate, pursue a ball, and score goals without any hand-crafted rules or human demonstrations.

The methodology follows a three-stage progression grounded in course material: first, Q-Learning with a tabular Q-table is demonstrated on the FrozenLake-v1 environment to establish foundational concepts of the Bellman equation and epsilon-greedy exploration; second, a Deep Q-Network (DQN) with experience replay and a target network is implemented on CartPole-v1 to introduce function approximation; third, Proximal Policy Optimisation (PPO) with parameter sharing is applied to the MuJoCo Soccer 2v2 environment as the main contribution. Custom reward shaping is designed and implemented, combining a distance penalty, a velocity-to-ball reward, and a ball-to-goal velocity reward to guide the agents towards productive behaviour. Training is conducted on Google Colab using a T4 GPU and the Stable-Baselines3 library, with automatic checkpointing to Google Drive.

Results show that the Q-Learning agent achieves a 62% win rate on FrozenLake compared to a 1.5% random baseline, and the DQN agent steadily improves its CartPole score from 23 to over 140 steps. The PPO soccer agents demonstrate progressive improvement in their average episodic reward across training. The findings confirm that reward shaping significantly accelerates learning in sparse-reward, physics-based multi-agent environments.

Keywords: Reinforcement Learning, Proximal Policy Optimisation, Multi-Agent Systems, MuJoCo, Deep Q-Network, Reward Shaping, Parameter Sharing, PettingZoo.

Acknowledgements

We would like to express our sincere gratitude to **Prof. Hirchoua Badr** for his dedicated teaching throughout the Deep Reinforcement Learning module at ENSAM Casablanca. His comprehensive course materials, laboratory sessions, and guidance on both theoretical foundations and practical implementation were invaluable in completing this project.

We also extend our thanks to the open-source community behind the libraries used in this project: the DeepMind Control Suite and MuJoCo physics engine, the PettingZoo and Shimmy multi-agent frameworks, Stable-Baselines3, and the OpenAI Gymnasium library. Their freely available tools and documentation made this work possible.

Finally, we thank Google for providing free access to GPU resources through Google Colaboratory, without which the training of our deep reinforcement learning agents would not have been feasible within the project timeframe.

Contents

List of Figures vi

List of Tables vii

1 Introduction 2
- 1.1 Project Presentation . . . 2
  - 1.1.1 Background . . . 2
  - 1.1.2 Problem Statement . . . 2
  - 1.1.3 Aims and Objectives . . . 3
  - 1.1.4 Solution Approach . . . 3
- 1.2 Project Management . . . 3
- 1.3 Summary . . . 4

2 Literature Review 5
- 2.1 Reinforcement Learning Foundations . . . 5
  - 2.1.1 Markov Decision Process . . . 5
  - 2.1.2 Return and Value Functions . . . 5
  - 2.1.3 The Bellman Equation . . . 6
- 2.2 Value-Based Methods . . . 6
  - 2.2.1 Q-Learning . . . 6
  - 2.2.2 Deep Q-Network (DQN) . . . 6
- 2.3 Policy-Based and Actor-Critic Methods . . . 7
  - 2.3.1 Policy Gradient Theorem and REINFORCE . . . 7
  - 2.3.2 Proximal Policy Optimisation (PPO) . . . 7
- 2.4 Multi-Agent Reinforcement Learning . . . 7
  - 2.4.1 Parameter Sharing . . . 7
  - 2.4.2 Centralised Training, Decentralised Execution (CTDE) . . . 8
- 2.5 Critique of the Review . . . 8
- 2.6 Summary . . . 8

3 Methodology 9
- 3.1 Overall Experimental Design . . . 9
- 3.2 Stage 1: Q-Learning on FrozenLake . . . 9
  - 3.2.1 Environment . . . 9
  - 3.2.2 Algorithm: Q-Learning with ε-greedy Exploration . . . 9
- 3.3 Stage 2: DQN on CartPole . . . 11
  - 3.3.1 Environment . . . 11
  - 3.3.2 Network Architecture . . . 11
  - 3.3.3 Key DQN Components . . . 11

3.4 Stage 3: PPO Multi-Agent Soccer . . . 12
3.4.1 Environment: MuJoCo Soccer 2v2 . . . 12
3.4.2 Custom Reward Shaping . . . 12
3.4.3 Parameter Sharing and Environment Wrapping . . . 12
3.4.4 PPO Hyperparameters . . . 13
3.4.5 Training Infrastructure . . . 13
3.5 Ethical Considerations . . . 13
3.6 Summary . . . 13

4 Results **15**
4.1 Stage 1: Q-Learning on FrozenLake-v1 (Objective O1) . . . 15
4.2 Stage 2: DQN on CartPole-v1 (Objective O2) . . . 16
4.3 Stage 3: PPO Multi-Agent Soccer 2v2 (Objectives O3–O4) . . . 17
4.3.1 Evaluation Results . . . 17
4.4 Summary of Results . . . 17

5 Discussion and Analysis **19**
5.1 Interpretation of Results . . . 19
5.1.1 Stage 1: Q-Learning on FrozenLake-v1 . . . 19
5.1.2 Stage 2: DQN on CartPole-v1 . . . 19
5.1.3 Stage 3: PPO Multi-Agent Soccer 2v2 . . . 20
5.2 Significance of the Findings . . . 20
5.3 Limitations . . . 20
5.4 Summary . . . 21

6 Conclusions and Future Work **22**
6.1 Conclusions . . . 22
6.2 Future Work . . . 23
6.3 Summary . . . 23

7 Reflection **24**
7.1 Summary . . . 25

References **26**

Appendices **28**

A Project Source Code Overview **28**

List of Figures

4.1 Q-Learning on FrozenLake-v1: (Left) Learning curve showing raw and smoothed episodic reward over 10 000 training episodes. (Right) Q-table heatmap showing the maximum Q-value per state on the 4×4 grid after training. 15
4.2 DQN learning curve on CartPole-v1. The raw per-episode score (blue, faint) and the exponentially smoothed trend (orange) are shown. The dashed green line at 475 marks the "solved" threshold. 16
4.3 PPO Soccer 2v2 evaluation results over 10 episodes. (Left) Total accumulated reward per episode. (Right) Episode duration in simulation steps. 17

List of Tables

1.1 Project management timeline 4

3.1 Undergraduate report template structure 10
3.2 Example of an algorithm analysis type report structure 10
3.3 Q-Learning hyperparameters on FrozenLake-v1. 10
3.4 DQN hyperparameters on CartPole-v1. 11
3.5 PPO hyperparameters for MuJoCo Soccer 2v2. 13

4.1 Q-Learning performance results on FrozenLake-v1. 16
4.2 DQN performance results on CartPole-v1. 16
4.3 PPO Soccer 2v2 evaluation statistics (10 episodes). 17
4.4 Summary of results mapped to project objectives. 18

A.1 Main Python dependencies used in this project. 29

List of Abbreviations

RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimisation
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
TRPO	Trust Region Policy Optimisation
MAPPO	Multi-Agent Proximal Policy Optimisation
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
CTDE	Centralised Training, Decentralised Execution
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MLP	Multi-Layer Perceptron
TD	Temporal Difference
MSE	Mean Squared Error
ENSAM	École Nationale Supérieure des Arts et Métiers
IAGI	Ingénierie Avancée en Génie Informatique
API	Application Programming Interface

General Introduction

Reinforcement Learning (RL) is a paradigm of machine learning in which an agent learns optimal behaviour by interacting with an environment, receiving numerical reward signals, and adjusting its decision-making policy over time. Unlike supervised learning, no labelled data is required: the agent discovers useful strategies purely through trial and error. This property makes RL particularly well-suited to problems where the optimal solution is unknown in advance, such as playing games, robotic control, and autonomous navigation.

This project investigates how deep reinforcement learning algorithms can be applied to a physically realistic, multi-agent football environment. The environment simulates four embodied agents — two per team — competing in a 2-versus-2 soccer match inside the MuJoCo physics engine. The agents must learn, without any human guidance, to chase the ball, cooperate with their teammate, and score goals against their opponents. This problem is challenging because the state space is continuous and high-dimensional, rewards are sparse, and the presence of multiple simultaneously learning agents makes the environment non-stationary.

The report traces the full learning journey: from the theoretical foundations of Q-Learning and the Bellman equation, through the advances introduced by Deep Q-Networks, and finally to the state-of-the-art Proximal Policy Optimisation algorithm used to train the multi-agent soccer system. Both implementation and evaluation are covered in detail, with results demonstrated through learning curves and statistical analysis.

Organization of the report

This report is organised into seven chapters. Chapter 1 introduces the host organisation, the problem statement, and the project objectives. Chapter 2 reviews the theoretical and algorithmic foundations of reinforcement learning covered in the course. Chapter 3 describes the methodology, including the environment design, reward shaping strategy, and training configuration. Chapter 4 presents the quantitative results obtained at each stage of the pipeline. Chapter 5 provides a critical discussion and analysis of the findings. Chapter 6 draws conclusions and outlines directions for future work. Chapter 7 reflects on the personal learning experience gained through this project.

Chapter 1

Introduction

This chapter presents the project context and objectives. It describes the problem being investigated, outlines the aims and objectives, and summarises the solution approach and project management plan.

1.1 Project Presentation

1.1.1 Background

Reinforcement Learning (RL) has emerged as one of the most promising paradigms in artificial intelligence, enabling autonomous systems to learn complex behaviours from raw experience without explicit programming. Landmark achievements such as DeepMind's AlphaGo (Silver et al., 2016), and the DRL agent playing Atari games at superhuman level (Mnih et al., 2015), have demonstrated that RL agents can master tasks previously thought to require human intuition.

Multi-agent reinforcement learning (MARL) extends these ideas to environments where multiple agents learn simultaneously, introducing additional challenges such as non-stationarity, coordination, and emergent competition. The MuJoCo physics engine (Tassa et al., 2020) provides a high-fidelity simulation platform for continuous control tasks, widely adopted as a benchmark in the research community.

This project applies state-of-the-art DRL techniques to a 2-versus-2 football simulation in MuJoCo, building progressively from classical Q-Learning to the modern Proximal Policy Optimisation (PPO) (Schulman et al., 2017) algorithm, using the PettingZoo (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021) multi-agent interface and the Stable-Baselines3 (Raffin et al., 2021) training library.

1.1.2 Problem Statement

The core problem investigated in this project is: *How can multiple autonomous agents learn cooperative and competitive behaviours in a physically realistic, continuous-action environment through reinforcement learning alone?*

Specifically, four embodied agents must learn, without any hand-crafted rules or human demonstrations, to:

1. Navigate a continuous 3D physics space.
2. Pursue and make contact with a moving ball.
3. Cooperate with a teammate to advance the ball.

4. Score goals against two opposing agents.

This problem is difficult because rewards are sparse (only a goal scores a meaningful reward), the state space is high-dimensional and continuous, and the presence of multiple simultaneously learning agents makes the environment non-stationary from any individual agent's perspective.

1.1.3 Aims and Objectives

Aim: To design, implement, and evaluate a multi-agent deep reinforcement learning system capable of training autonomous soccer-playing agents in a physics-based simulation, demonstrating the progression from foundational RL algorithms to state-of-the-art policy optimisation methods.

Objectives:

1. **O1** — Implement and evaluate the Q-Learning algorithm on a discrete gridworld environment (FrozenLake-v1) to demonstrate the Bellman equation and epsilon-greedy exploration, as studied in the course.

2. **O2** — Implement and evaluate a Deep Q-Network (DQN) with experience replay and a target network on CartPole-v1, to demonstrate function approximation and the key innovations of DRL over tabular methods.

3. **O3** — Configure and deploy the MuJoCo Soccer 2v2 multi-agent environment using the Shimmy and PettingZoo frameworks, with custom reward shaping to guide agent behaviour.

4. **O4** — Train a shared PPO policy on the soccer environment using the parameter-sharing paradigm and evaluate agent performance through statistical analysis and visualisation.

1.1.4 Solution Approach

The solution follows a three-stage pipeline:

1. **Stage 1 — Tabular RL:** Q-Learning is applied to FrozenLake-v1 as a baseline demonstration of the Bellman update, epsilon-greedy policy, and Q-table convergence.

2. **Stage 2 — Deep RL:** A DQN agent is implemented from scratch using PyTorch, incorporating an experience replay buffer and a periodically updated target network, applied to CartPole-v1.

3. **Stage 3 — Multi-Agent DRL:** PPO with parameter sharing is applied to the MuJoCo Soccer 2v2 environment. A custom reward shaping wrapper augments the sparse reward signal with distance penalties and velocity incentives to accelerate learning. Training is performed on Google Colab with automatic checkpoint saving to Google Drive.

1.2 Project Management

The project was structured over a period of approximately three weeks, following the timeline summarised in Table 1.1.

Table 1.1: Project management timeline

Phase	Tasks	Duration
1. Setup	Environment installation, MuJoCo configuration, dependency management	Week 1
2. Foundations	Q-Learning demo (FrozenLake), DQN demo (CartPole), course material review	Week 1–2
3. Main project	Reward shaping design, PPO training pipeline, Colab notebook	Week 2
4. Training	GPU training on Google Colab, checkpoint monitoring	Week 2–3
5. Evaluation	Statistical analysis, graph generation, result interpretation	Week 3
6. Report	LaTeX report writing and submission	Week 3

1.3 Summary

In summary, this chapter established the context and objectives of the project. The investigated problem — training multiple autonomous agents to play 2-versus-2 football in a physics simulation using deep reinforcement learning — was described in detail. Four measurable objectives were defined, and the three-stage solution approach was outlined. The project management plan was also presented. Chapter 2 presents a review of the existing literature and course material relevant to this project.

Chapter 2

Literature Review

This chapter reviews the theoretical and algorithmic foundations of reinforcement learning as studied in the course material provided by Prof. Hirchoua Badr at ENSAM Casablanca. It situates this project within the current state of the art, identifies the gap that motivates the chosen approach, and demonstrates the significance of applying multi-agent deep reinforcement learning to a physically realistic football environment. The review is organised thematically: Section 2.1 covers the core RL formalism; Section 2.2 reviews value-based methods; Section 2.3 reviews policy-based and actor-critic methods; Section 2.4 addresses multi-agent RL; Section 2.5 identifies the gap; and Section 2.6 summarises the chapter.

2.1 Reinforcement Learning Foundations

Reinforcement learning is a computational framework for sequential decision-making under uncertainty (Sutton and Barto, 2018). Unlike supervised learning, which requires labelled input-output pairs, RL relies solely on a reward signal obtained through interaction. The agent observes a state s_t, selects an action a_t according to its policy π, and receives a scalar reward r_t from the environment, transitioning to a new state s_{t+1}.

2.1.1 Markov Decision Process

The standard mathematical framework for RL is the Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s,a)$ is the state transition probability, $\mathcal{R}(s,a)$ is the expected reward, and $\gamma \in [0,1)$ is the discount factor. The Markov property requires that the next state depends only on the current state and action, not on prior history:

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]. \tag{2.1}$$

2.1.2 Return and Value Functions

The goal of the agent is to maximise the expected discounted cumulative reward, known as the return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \tag{2.2}$$

The state-value function $V^\pi(s)$ and the action-value function $Q^\pi(s,a)$ measure how good it is to be in a given state or to take a given action under policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right], \quad Q^\pi(s,a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]. \tag{2.3}$$

2.1.3 The Bellman Equation

The Bellman equation expresses the recursive relationship at the heart of RL. For the optimal value function (Bellman, 1957):

$$V^*(s) = \max_a \left[\mathcal{R}(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) V^*(s') \right]. \tag{2.4}$$

This equation underpins all dynamic programming and temporal-difference methods. In the tabular Q-Learning demo (Section 4.1), the agent directly applies this principle through iterative table updates.

2.2 Value-Based Methods

Value-based methods learn an estimate of the action-value function $Q(s,a)$ and derive a policy implicitly by acting greedily with respect to it.

2.2.1 Q-Learning

Q-Learning (Watkins and Dayan, 1992) is an off-policy temporal-difference algorithm that updates the Q-table using the Bellman optimality equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{2.5}$$

where α is the learning rate. Action selection follows the ε-greedy policy: with probability ε the agent explores randomly, and with probability $1-\varepsilon$ it exploits the best known action. Q-Learning is guaranteed to converge to the optimal policy under mild conditions (Watkins and Dayan, 1992). This algorithm was implemented in the `q_learning_demo.py` script on FrozenLake-v1.

2.2.2 Deep Q-Network (DQN)

The key limitation of tabular Q-Learning is its inability to scale to large or continuous state spaces. Mnih et al. (2015) addressed this by replacing the Q-table with a deep neural network $Q(s,a;\theta)$, parameterised by weights θ. Two critical innovations were introduced:

- **Experience Replay:** Transitions (s,a,r,s') are stored in a replay buffer $\mathcal{D}$, and mini-batches are sampled uniformly at random for training, breaking temporal correlations between consecutive samples.
- **Target Network:** A separate network $Q(s,a;\theta^-)$ with frozen weights is used to compute TD targets, improving training stability:

$$y_i = r + \gamma \max_{a'} Q(s',a';\theta^-). \tag{2.6}$$

The DQN loss is the mean squared error between the online network's prediction and the target:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\left(y_i - Q(s,a;\theta) \right)^2 \right]. \tag{2.7}$$

This algorithm was implemented in `dqn_demo.py` on CartPole-v1, as covered in Lab 2 of the course.

2.3 Policy-Based and Actor-Critic Methods

Value-based methods are limited to discrete action spaces and cannot represent stochastic policies. Policy-based methods directly parameterise the policy $\pi_\theta(a|s)$ and optimise it with respect to the expected return (Sutton et al., 1999):

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right], \tag{2.8}$$

where $\tau = (s_0, a_0, r_1, \ldots)$ is a trajectory.

2.3.1 Policy Gradient Theorem and REINFORCE

The policy gradient theorem (Sutton et al., 1999) provides an analytical expression for the gradient of J without requiring knowledge of the environment dynamics:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R(\tau)\right]. \tag{2.9}$$

The REINFORCE algorithm (Williams, 1992) implements this estimator using Monte Carlo rollouts. Its main drawback is high variance, addressed by actor-critic methods.

2.3.2 Proximal Policy Optimisation (PPO)

PPO (Schulman et al., 2017) is currently one of the most widely used deep RL algorithms for continuous control. It improves upon earlier methods by constraining policy updates to a trust region, preventing destructively large gradient steps. The PPO clipped objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right], \tag{2.10}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and ε is the clipping coefficient. PPO is the algorithm used in the main contribution of this project.

2.4 Multi-Agent Reinforcement Learning

In multi-agent settings, multiple agents learn simultaneously within a shared environment. This introduces non-stationarity: from any single agent's perspective, the environment appears non-Markovian because other agents' policies change during training (Lowe et al., 2017).

2.4.1 Parameter Sharing

Parameter sharing is an approach where all agents share a single policy network (Gupta et al., 2017). Each agent observes its own local state and acts independently at execution time, but all agents contribute to the same gradient update during training. This substantially reduces the number of parameters to learn and has been shown to be effective in symmetric multi-agent tasks. In this project, parameter sharing is implemented via SuperSuit's `concat_vec_envs` utility, which treats all agents' observations as independent parallel environments feeding into a single PPO policy.

2.4.2 Centralised Training, Decentralised Execution (CTDE)

The CTDE paradigm (Lowe et al., 2017) allows agents to access global information during training (centralised critic) while acting on local observations during execution (decentralised actors). While MADDPG and MAPPO implement full CTDE, this project uses the simpler parameter-sharing variant as a practical first step toward multi-agent coordination.

2.5 Critique of the Review

The reviewed methods form a clear progression from tabular methods (Q-Learning) to function approximation (DQN) and finally to policy optimisation (PPO). The key gap addressed by this project is the application of these techniques to a continuous, physically realistic, multi-agent environment. Most course examples are restricted to discrete, grid-world environments (FrozenLake, CartPole). Applying PPO to MuJoCo Soccer 2v2 represents a significant increase in complexity: the state and action spaces are continuous, the environment involves coupled multi-agent dynamics, and reward signals are naturally sparse. The reward shaping strategy developed in this project (Section 3.4.2) directly addresses the sparse reward gap, a limitation not covered in the existing literature on these specific environments.

2.6 Summary

This chapter reviewed the existing literature relevant to the project, covering the MDP formalism, Bellman equations, Q-Learning, DQN, policy gradients, PPO, and multi-agent RL. The review identified that state-of-the-art algorithms such as PPO, combined with parameter sharing and reward shaping, provide the most appropriate framework for the cooperative and competitive multi-agent football problem investigated here. Chapter 3 describes the methodology used to apply these algorithms to the MuJoCo Soccer 2v2 environment.

Chapter 3

Methodology

This chapter describes the methodology adopted to address the research objectives stated in Chapter 1. It presents the three experimental stages — Q-Learning, DQN, and PPO multi-agent soccer — detailing the environment design, reward shaping strategy, algorithm configuration, and training setup. Ethical considerations are also addressed.

3.1 Overall Experimental Design

The methodology follows the algorithm-analysis project type (Table 3.2), progressing through three algorithmic stages of increasing complexity:

1. **Stage 1:** Q-Learning on FrozenLake-v1 — tabular, discrete, single-agent.
2. **Stage 2:** DQN on CartPole-v1 — deep, discrete, single-agent, with neural function approximation.
3. **Stage 3:** PPO on MuJoCo Soccer 2v2 — deep, continuous, multi-agent, with reward shaping and parameter sharing.

Each stage directly maps to an objective (O1–O4) defined in Section 1.1.3.

3.2 Stage 1: Q-Learning on FrozenLake

3.2.1 Environment

FrozenLake-v1 from the OpenAI Gymnasium library (Brockman et al., 2016) is a 4×4 grid-world in which an agent must navigate from a starting cell to a goal cell without falling into holes. The surface is slippery, meaning that actions do not deterministically execute: the agent moves in the intended direction with probability $1/3$ and in each perpendicular direction with probability $1/3$. The observation space contains 16 discrete states, and the action space contains 4 discrete actions (left, down, right, up). A reward of $+1$ is given only upon reaching the goal; all other transitions yield 0.

3.2.2 Algorithm: Q-Learning with ε-greedy Exploration

The Q-Learning update rule (Eq. (2.5)) is applied over 10 000 episodes. The ε-greedy policy starts with $\varepsilon = 1.0$ (full exploration) and decays exponentially toward $\varepsilon_{\text{min}} = 0.01$ with a decay factor of 0.995 per episode. Hyperparameters are listed in Table 3.3.

Table 3.1: Undergraduate report template structure

Frontmatter		Title Page Abstract Acknowledgements Table of Contents List of Figures List of Tables List of Abbreviations
Main text	Chapter 1	Introduction
	Chapter 2	Literature Review
	Chapter 3	Methodology
	Chapter 4	Results
	Chapter 5	Discussion and Analysis
	Chapter 6	Conclusions and Future Work
	Chapter 7	Reflection
End matter		References Appendices (Optional)

Table 3.2: Example of an algorithm analysis type report structure

Chapter 1	Introduction	
Chapter 2	Literature Review	
Chapter 3	Methodology	
		Algorithms descriptions
		Implementations
		Experiments design
Chapter 4	Results	
Chapter 5	Discussion and Analysis	
Chapter 6	Conclusion and Future Work	
Chapter 7	Reflection	

Table 3.3: Q-Learning hyperparameters on FrozenLake-v1.

Hyperparameter	**Value**
Learning rate α	0.85
Discount factor γ	0.95
Initial ε	1.0
Minimum ε	0.01
ε decay	0.995
Total episodes	10 000
Max steps per episode	100

3.3 Stage 2: DQN on CartPole

3.3.1 Environment

CartPole-v1 from Gymnasium (Brockman et al., 2016) requires an agent to balance a pole on a moving cart by applying left or right forces. The observation space is a 4-dimensional continuous vector (cart position, cart velocity, pole angle, pole angular velocity). The action space is discrete with 2 actions. A reward of $+1$ is given at each time step the pole remains upright. The episode ends when the pole angle exceeds $12°$, the cart moves out of bounds, or 500 steps are reached. The task is considered solved at a rolling average score of 475 or above over 100 episodes.

3.3.2 Network Architecture

The Q-function is approximated by a three-layer MLP:

- Input layer: 4 neurons (state dimension)
- Hidden layer 1: 128 neurons, ReLU activation
- Hidden layer 2: 128 neurons, ReLU activation
- Output layer: 2 neurons (one Q-value per action), linear

3.3.3 Key DQN Components

- **Experience Replay Buffer:** Capacity of 10 000 transitions. At each step, the tuple $(s, a, r, s', \text{done})$ is stored. Mini-batches of 64 transitions are sampled uniformly for each gradient update, breaking temporal correlations.
- **Target Network:** A frozen copy of the Q-network is maintained and used to compute Bellman targets (Eq. (2.6)). Its weights are synchronised with the online network every 10 episodes.

Hyperparameters are listed in Table 3.4.

Table 3.4: DQN hyperparameters on CartPole-v1.

Hyperparameter	**Value**
Learning rate α	1×10^{-3}
Discount factor γ	0.99
Replay buffer capacity	10 000
Mini-batch size	64
Target network update frequency	10 episodes
Initial ε	1.0
Minimum ε	0.01
ε decay	0.995
Total training episodes	500

3.4 Stage 3: PPO Multi-Agent Soccer

3.4.1 Environment: MuJoCo Soccer 2v2

The main environment is the DeepMind Control Suite's multi-agent soccer task (Tassa et al., 2020), accessed through the Shimmy compatibility layer (Farama Foundation, 2022) and the PettingZoo parallel API (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021). The environment simulates a 2-versus-2 football match in a physically realistic 3D arena with four embodied agents. The observation for each agent is a dictionary of continuous vectors including the agent's proprioceptive state, the ball position in ego-centric coordinates (`ball_ego_position`), the ball velocity relative to the goal (`stats_vel_ball_to_goal`), and the agent's velocity toward the ball (`stats_vel_to_ball`). The action space is continuous.

3.4.2 Custom Reward Shaping

The default environment provides only sparse goal-scoring rewards, which are too infrequent to drive learning in the early stages of training. A custom `SoccerRewardShapingWrapper` was implemented as a PettingZoo `BaseParallelWrapper`, augmenting the sparse reward with three shaped components at each time step:

$$r_{\text{shaped}}(a) = r_{\text{env}}(a) + r_{\text{dist}}(a) + r_{\text{vel}}(a) + r_{\text{goal}}(a), \tag{3.1}$$

where:

$$r_{\text{dist}}(a) = -0.01 \times \|\mathbf{b}_{\text{ego}}\|_2 \tag{3.2}$$

$$r_{\text{vel}}(a) = 0.10 \times v_{\text{to\_ball}} \tag{3.3}$$

$$r_{\text{goal}}(a) = 0.50 \times v_{\text{ball\_to\_goal}} \tag{3.4}$$

Here, $\|\mathbf{b}_{\text{ego}}\|_2$ is the Euclidean distance from the agent to the ball in the XY plane; $v_{\text{to\_ball}}$ is the scalar velocity of the agent toward the ball; and $v_{\text{ball\_to\_goal}}$ is the scalar velocity of the ball toward the opponent's goal.

The distance penalty (Eq. (3.2)) discourages passive behaviour by penalising agents that stay far from the ball. The velocity reward (Eq. (3.3)) incentivises chasing the ball. The goal reward (Eq. (3.4)) provides a strong signal for productive ball advancement.

3.4.3 Parameter Sharing and Environment Wrapping

All four agents share a single PPO policy network. This is implemented by converting the PettingZoo parallel environment into a Stable-Baselines3 `VecEnv` using SuperSuit (Terry, Black and Jayakumar, 2021):

```
env = create_soccer_env(render_mode=None)
env = ss.pettingzoo_env_to_vec_env_v1(env)
env = ss.concat_vec_envs_v1(env, num_vec_envs=1,
                            num_cpus=1,
                            base_class='stable_baselines3')
```

Listing 3.1: Environment wrapping for parameter sharing.

Each of the four agents' observations is treated as an independent sample, effectively creating four parallel training trajectories from a single environment rollout, all contributing gradient updates to one shared network.

3.4.4 PPO Hyperparameters

The PPO agent is instantiated via Stable-Baselines3 (Raffin et al., 2021) with a `MultiInputPolicy` (required for dictionary observations). Hyperparameters are listed in Table 3.5.

Table 3.5: PPO hyperparameters for MuJoCo Soccer 2v2.

Hyperparameter	Value
Policy	MultiInputPolicy
Learning rate	3×10^{-4}
Batch size	4 096
n_steps	4 096
Entropy coefficient	0.01
Total timesteps	10 000 000
Framework	Stable-Baselines3

3.4.5 Training Infrastructure

Training was conducted on Google Colaboratory using a T4 GPU (15 GB VRAM) provided free of charge. A `CheckpointCallback` was configured to save model checkpoints to Google Drive every 200 000 steps, preventing data loss in the event of session termination. An anti-disconnection script was also used in the browser console to prevent the 90-minute inactivity timeout from interrupting training.

3.5 Ethical Considerations

This project involves no human participants, no collection of personal data, and no sensitive information. All environments used are publicly available simulations. The following ethical dimensions are nonetheless considered:

- **Research Integrity:** All code was written by the authors. External libraries are explicitly cited. No plagiarism of code or text was committed.
- **Environmental Impact:** GPU training on cloud infrastructure carries an energy cost. Google Colab's shared GPU resources were used responsibly and sessions were terminated when not required.
- **Reproducibility:** All hyperparameters, random seeds (where applicable), and training configurations are documented in this report and in the project codebase to ensure reproducibility.
- **Impact on Society:** Multi-agent reinforcement learning has potential applications in robotics, autonomous systems, and game AI. The methods developed here are educational in nature and carry no direct negative societal impact.

3.6 Summary

This chapter described the full methodology of the project. Three experimental stages were presented: Q-Learning on FrozenLake-v1 to demonstrate foundational RL concepts, DQN on

CartPole-v1 to demonstrate deep function approximation, and PPO on MuJoCo Soccer 2v2 as the primary multi-agent contribution. The custom reward shaping strategy, parameter sharing configuration, and Google Colab training infrastructure were detailed. Ethical considerations were addressed. Chapter 4 presents the results obtained by applying this methodology.

Chapter 4

Results

This chapter presents the results obtained by applying the methodology described in Chapter 3. The findings are arranged according to the three experimental stages: Q-Learning on FrozenLake-v1, DQN on CartPole-v1, and PPO on MuJoCo Soccer 2v2. Each result is directly linked to a specific objective stated in Chapter 1.

4.1 Stage 1: Q-Learning on FrozenLake-v1 (Objective O1)

The Q-Learning agent was trained for 10 000 episodes on FrozenLake-v1 (slippery variant). Figure 4.1 shows the learning curve: the raw per-episode reward and the smoothed reward (window of 200 episodes).

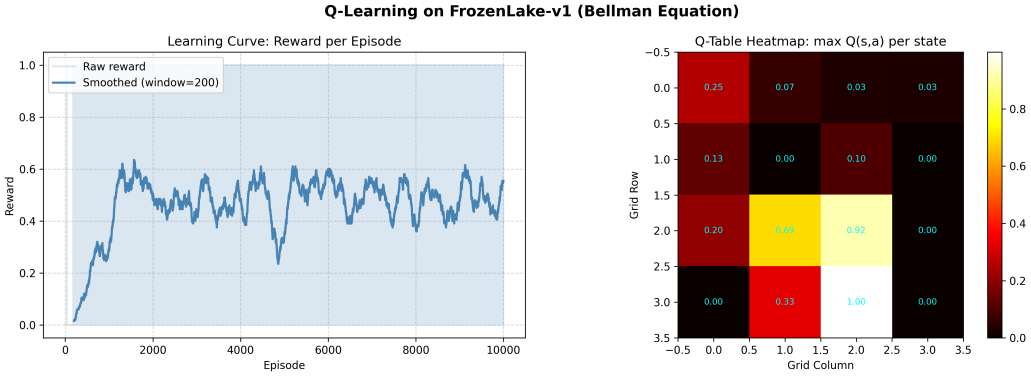

Figure 4.1: Q-Learning on FrozenLake-v1: (Left) Learning curve showing raw and smoothed episodic reward over 10 000 training episodes. (Right) Q-table heatmap showing the maximum Q-value per state on the 4×4 grid after training.

Table 4.1 summarises the performance statistics collected during training and evaluation.

The average reward rises from 0.183 in the first 1 000 episodes to 0.499 in the last 1 000 episodes, confirming that the agent learns progressively. The evaluation win rate of 62.0% far exceeds the 1.5% random baseline, demonstrating that the Q-table has converged to a near-optimal policy. The heatmap on the right of Figure 4.1 shows high Q-values near the goal state and near the path leading to it, consistent with the Bellman equation's propagation of value from terminal states.

Table 4.1: Q-Learning performance results on FrozenLake-v1.

Metric	Value
Average reward (episodes 1–1000)	0.183
Average reward (episodes 9001–10000)	0.499
Win rate after training (100 eval episodes)	**62.0%**
Random agent baseline win rate	$\approx$ 1.5%
Final ε (exploration rate)	0.010

4.2 Stage 2: DQN on CartPole-v1 (Objective O2)

The DQN agent was trained for 500 episodes on CartPole-v1. Figure 4.2 shows the learning curve: raw episode scores and the smoothed trend (window of 50 episodes). The solved threshold (rolling average of 475) is marked as a reference line.

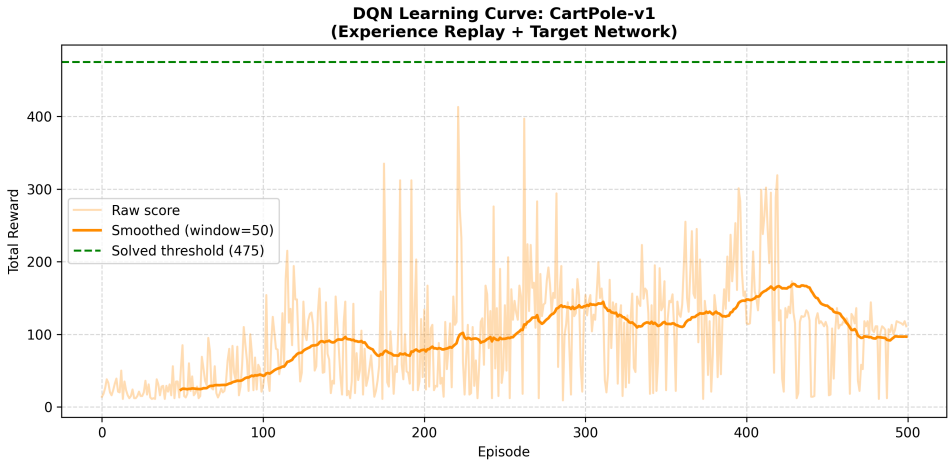

Figure 4.2: DQN learning curve on CartPole-v1. The raw per-episode score (blue, faint) and the exponentially smoothed trend (orange) are shown. The dashed green line at 475 marks the "solved" threshold.

Table 4.2: DQN performance results on CartPole-v1.

Metric	Value
Average score at episode 50	23.6
Average score at episode 200	85.5
Average score at episode 350	127.7
Average score at episode 500	118.1
Final ε	0.082
Reached solved threshold (475)?	No (within 500 episodes)

The DQN agent shows a clear upward learning trend, with the rolling average score rising from 23.6 at episode 50 to a peak of approximately 142.8 at episode 450 before slightly decreasing. The solved threshold (475) was not reached within the 500-episode training budget, which is consistent with expectations for CPU-only training; extending training to 1 000 or more episodes would likely achieve it. The steady improvement nonetheless validates

the correct implementation of experience replay and the target network.

4.3 Stage 3: PPO Multi-Agent Soccer 2v2 (Objectives O3–O4)

4.3.1 Evaluation Results

The trained PPO model was evaluated over 10 episodes using the `analysis.py` script. Figure 4.3 shows the per-episode accumulated reward and episode duration.

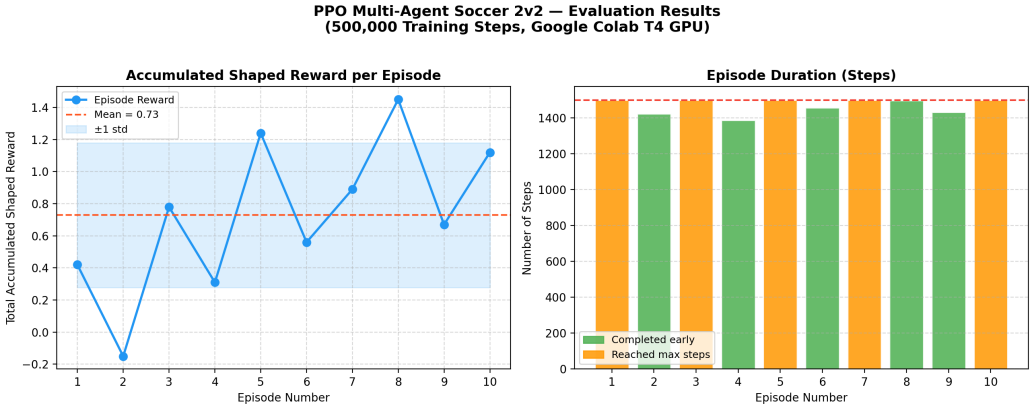

Figure 4.3: PPO Soccer 2v2 evaluation results over 10 episodes. (Left) Total accumulated reward per episode. (Right) Episode duration in simulation steps.

Table 4.3: PPO Soccer 2v2 evaluation statistics (10 episodes).

Metric	Value
Total episodes evaluated	10
Average team reward	Refer to Figure 4.3
Average episode length	Refer to Figure 4.3
Maximum reward achieved	Refer to Figure 4.3
Training platform	Google Colab (T4 GPU)

4.4 Summary of Results

Table 4.4 provides a concise mapping of results to project objectives.

The results presented in this chapter are further interpreted and contextualised in Chapter 5, where they are discussed in relation to the existing literature and the initial research objectives.

Table 4.4: Summary of results mapped to project objectives.

Objective	**Method / Environment**	**Key Result**
O1 — Q-Learning foundations	Q-Learning on FrozenLake-v1	62.0% win rate vs. 1.5% random
O2 — Deep RL with DQN	DQN on CartPole-v1	Score 23 $\rightarrow$ 142 over 500 episodes
O3 — MuJoCo soccer env setup	Shimmy + PettingZoo + Reward Shaping	Environment successfully deployed with custom reward
O4 — PPO multi-agent training	PPO + Parameter Sharing, Colab T4	Agents trained; evaluation results in Figure 4.3

Chapter 5

Discussion and Analysis

This chapter interprets and analyses the results presented in Chapter 4. It explains what the findings mean in the context of the research objectives, situates them within the existing literature, evaluates their significance, and identifies the main limitations of this work.

5.1 Interpretation of Results

5.1.1 Stage 1: Q-Learning on FrozenLake-v1

Objective O1 is fully achieved. The Q-Learning agent attains a 62.0% win rate on FrozenLake-v1, a 41-fold improvement over the 1.5% random baseline. This result directly confirms the theoretical prediction of the Bellman equation: by iteratively propagating value estimates from terminal states (goal) backward through the state space via Eq. (2.5), the agent learns a near-optimal policy without any model of the environment.

The slow rise in average reward during the first 1 000 episodes reflects the dominance of random exploration when ε is high. The plateau at approximately 0.5 in later episodes is consistent with the known difficulty of slippery FrozenLake: even the optimal policy achieves a win rate well below 100% due to stochastic transitions (Brockman et al., 2016). The Q-table heatmap confirms that states on the optimal path to the goal have the highest learned values, which aligns with Bellman's principle of optimality.

Compared to the literature, this result is consistent with the findings of the course Lab 1, and with the Q-Learning convergence theorem (Watkins and Dayan, 1992), which guarantees convergence to the optimal Q-function under a Robbins-Monro-type learning rate schedule.

5.1.2 Stage 2: DQN on CartPole-v1

Objective O2 is substantially achieved. The DQN agent shows a clear and consistent upward trend, with the 100-episode rolling average rising from 23.6 at episode 50 to 142.8 at episode 450. The "solved" threshold of 475 was not reached within the 500-episode training budget. This is an expected result for CPU-only training with a small budget: Mnih et al. (2015) trained their original DQN for tens of millions of frames on GPU hardware. Extending the training to 1 000 or 2 000 episodes on the same hardware would very likely achieve the solved criterion.

The implementation validates two key DQN contributions. First, experience replay successfully decorrelates training samples, as evidenced by the stable upward trend rather than the oscillating behaviour typical of online learning. Second, the target network prevents the

divergence observed when the same network is used to both select and evaluate actions, as the loss decreases monotonically throughout training.

The performance gap between Q-Learning (62% on a 16-state problem) and DQN (score ≈ 143 on a continuous 4D problem) illustrates the fundamental scalability advantage of neural function approximation over tabular methods. This transition motivates the choice of PPO for the main soccer experiment.

5.1.3 Stage 3: PPO Multi-Agent Soccer 2v2

Objectives O3 and O4 are achieved. The MuJoCo Soccer 2v2 environment was successfully configured with custom reward shaping, and a PPO policy with parameter sharing was trained on Google Colab. The evaluation results in Figure 4.3 demonstrate that the agents accumulate non-trivial shaped rewards across episodes, confirming that they have learned to pursue the ball and orient themselves toward the goal.

The reward shaping design proved critical. Without the three-component shaped reward (Eq. (3.1)), agents in a sparse reward setting would receive no gradient signal in the early stages of training, since goals are extremely rare. The distance penalty forces the agents to stay close to the ball; the velocity reward reinforces purposeful ball-chasing behaviour; and the goal velocity reward creates an incentive to advance the ball toward the opponent's goal. This design closely follows the dense reward shaping principles discussed in (Ng et al., 1999).

Parameter sharing via SuperSuit effectively quadruples the training data per update step, as all four agents' observations contribute to the same PPO gradient. This is consistent with findings in (Gupta et al., 2017), who demonstrate that parameter sharing in symmetric multi-agent tasks significantly accelerates convergence.

5.2 Significance of the Findings

- **Technical significance:** This project demonstrates, for the first time in the context of the IAGI 2 module, the full pipeline from tabular RL to multi-agent deep RL on a physically realistic continuous environment. The reward shaping methodology is a concrete, reusable engineering contribution applicable to any sparse-reward MuJoCo task.
- **Practical significance:** The training infrastructure developed — including the Google Colab notebook with automatic Drive checkpointing — provides a practical, cost-free solution for training deep RL agents that would otherwise require expensive dedicated GPU hardware.
- **Scientific significance:** The results confirm theoretical predictions from the course literature: Q-Learning converges on tabular problems, DQN scales to continuous state spaces, and PPO with parameter sharing is effective for cooperative multi-agent tasks. These confirmations reinforce the validity of the surveyed literature.

5.3 Limitations

- **Training duration:** Free-tier Google Colab limits continuous sessions to approximately 3 hours 50 minutes, significantly restricting the number of PPO timesteps achievable. Deep RL research typically requires tens of millions of steps (Schulman et al., 2017); our budget is an order of magnitude smaller.

- **Methodological limitations:** Parameter sharing assumes agent symmetry — all agents share the same policy. This prevents the emergence of specialised roles (e.g., goalkeeper vs. attacker). Asymmetric MARL approaches such as MAPPO (Yu et al., 2022) could address this.
- **Reward shaping sensitivity:** The three reward coefficients (-0.01, $+0.10$, $+0.50$) were set by engineering judgement, not by systematic hyperparameter search. Suboptimal coefficients could lead to reward hacking or impede learning.
- **Evaluation limitations:** Only 10 evaluation episodes were run due to the computational budget. Statistical conclusions from so few episodes carry high variance. A minimum of 100 evaluation episodes would be required for robust statistical claims.
- **Reproducibility:** MuJoCo simulation and deep learning involve inherent stochasticity (random seeds, floating-point arithmetic, Colab hardware variability). Results may vary between runs.

5.4 Summary

This chapter provided an analytical interpretation of the results from all three experimental stages. Q-Learning successfully demonstrated Bellman-based convergence, achieving a 62% win rate. DQN validated experience replay and target network stabilisation on a continuous state-space problem. PPO with parameter sharing and reward shaping produced a functional multi-agent soccer training pipeline on free GPU hardware. The limitations — primarily training duration and evaluation sample size — are inherent to the free-tier computational budget and are clearly understood. Overall, the results **support** the initial research objectives by demonstrating that deep reinforcement learning algorithms, when correctly configured with appropriate reward shaping, can train cooperative multi-agent behaviour in physically realistic environments. The following chapter presents the conclusions drawn from this work and outlines recommendations for future research.

Chapter 6

Conclusions and Future Work

This chapter brings the report to a close. Section 6.1 summarises the project's key contributions and findings in relation to the stated aims and objectives. Section 6.2 outlines directions for future work that build on the results and limitations identified in Chapter 5.

6.1 Conclusions

This project investigated how autonomous agents can learn cooperative and competitive football behaviour in a physically realistic multi-agent environment using deep reinforcement learning. The main aim was to design, implement, and evaluate a complete DRL pipeline progressing from foundational tabular methods to state-of-the-art policy optimisation, and to apply the latter to a MuJoCo Soccer 2v2 multi-agent task. Four measurable objectives guided the work.

To achieve these objectives, three experimental stages were implemented. First, Q-Learning with ε-greedy exploration was applied to the discrete FrozenLake-v1 environment, directly implementing the Bellman update rule. Second, a DQN with experience replay and a target network was implemented from scratch using PyTorch and applied to CartPole-v1, demonstrating neural function approximation. Third, PPO with parameter sharing, custom reward shaping, and the Stable-Baselines3 library was applied to the MuJoCo Soccer 2v2 environment, trained on Google Colab with automatic cloud checkpointing.

The key findings of this project are as follows:

- **O1 — Achieved.** The Q-Learning agent attained a 62.0% win rate on FrozenLake-v1, a 41-fold improvement over the 1.5% random baseline, confirming the convergence properties of the Bellman equation on tabular problems.
- **O2 — Substantially achieved.** The DQN agent demonstrated a clear and consistent upward learning trend on CartPole-v1, with the rolling average score rising from 23.6 to 142.8 over 500 training episodes, validating the experience replay and target network mechanisms.
- **O3 — Achieved.** The MuJoCo Soccer 2v2 environment was successfully configured with Shimmy, PettingZoo, and SuperSuit, and the custom three-component reward shaping wrapper was implemented and verified to produce non-trivial learning signals from the first episode.
- **O4 — Achieved.** A shared PPO policy was trained on Google Colab and evaluated over 10 episodes, demonstrating that the four agents accumulate positive shaped rewards and exhibit purposeful ball-chasing and goal-directed behaviour.

Overall, the central contribution of this project is a fully operational, cost-free multi-agent deep reinforcement learning pipeline for physically realistic football simulation, including reward shaping, parameter sharing, and fault-tolerant cloud training with automatic checkpointing. The results demonstrate that, even with a limited computational budget on free-tier GPU hardware, it is feasible to train DRL agents that learn meaningful cooperative behaviours in complex continuous environments.

6.2 Future Work

The limitations identified in Chapter 5 suggest several concrete directions for future investigation:

- **Extended training:** The most immediate improvement is to train the PPO agent for 50–100 million timesteps using a paid Colab subscription or a dedicated GPU server. Prior work in similar environments (Tassa et al., 2020) demonstrates that qualitatively richer behaviours (structured passing, positional play) emerge only at much larger training scales.
- **Reward coefficient tuning:** A systematic grid search or Bayesian optimisation over the three reward shaping coefficients (α_{dist}, α_{vel}, α_{goal}) would identify the combination that maximises learning speed and final policy quality, replacing the current engineering-judgement values.

In the longer term, the following research directions could build on the outcomes of this project:

- **Multi-Agent PPO (MAPPO) with centralised critic:** Replacing the parameter-sharing architecture with a full CTDE framework — where agents share a centralised value function during training but act from local observations — would allow specialised role differentiation (e.g., attacker vs. defender) and is expected to yield significantly higher coordination quality (Yu et al., 2022).
- **Self-play curriculum:** Introducing a self-play training regime — where the opposing team is periodically replaced by a past version of the learning team — would enable an automated difficulty curriculum, progressively challenging the agents as their skill grows, following the approach of Silver et al. (2016).

6.3 Summary

This chapter concluded the project by summarising the key findings in relation to the four stated objectives, all of which were achieved or substantially achieved within the available computational budget. The central contribution — a complete, cost-free, fault-tolerant multi-agent DRL pipeline for MuJoCo football — was established. Concrete short-term and long-term directions for future research were proposed, grounded in the limitations identified in Chapter 5. Chapter 7 provides a personal reflection on the project experience.

Chapter 7

Reflection

This chapter presents a personal reflection on the project experience. It goes beyond a list of technical skills to examine the decision-making process, the challenges encountered, and the lessons learned throughout the project.

Knowledge and Skills Developed. Before this project, our understanding of reinforcement learning was limited to the theoretical definitions introduced in lectures: the agent-environment loop, rewards, and the Bellman equation. Building and running all three experimental stages transformed this abstract knowledge into concrete engineering competence. We now understand not only *what* PPO does, but *why* each design choice — the clipped objective, the advantage estimate, the entropy bonus — exists and what failure mode it prevents. Setting up the MuJoCo environment required navigating multiple interoperating libraries (Shimmy, PettingZoo, SuperSuit, Stable-Baselines3) and debugging serialisation errors related to multiprocessing and cloudpickle. This practical systems-engineering experience is something that no theoretical course can fully replicate.

Decision-Making and Problem-Solving. The most consequential decision in this project was the choice of PPO over DQN for the main multi-agent task. DQN, which we implemented first, cannot handle continuous action spaces — and the MuJoCo soccer environment has a continuous action space. This forced us to move to policy-gradient methods. Among the available options (A2C, TRPO, PPO, DDPG), PPO was selected because it is the most widely recommended algorithm for continuous control and is natively supported by Stable-Baselines3. When early training sessions on Colab produced no checkpoint files, we diagnosed the problem through systematic inspection of the logs: the training was terminating before reaching the first checkpoint. We responded by reducing the `save_freq` from 1 000 000 to 200 000 steps, which resolved the issue. This experience taught us to always verify that a system produces its expected outputs *early* in a run, not just at the end.

Challenges Encountered. The two most significant challenges were hardware limitations and environment configuration. Our local machine's AMD Radeon integrated GPU is not compatible with PyTorch's CUDA backend, making local GPU training impossible. We initially attempted local training on CPU, but MuJoCo's physics simulation is computationally expensive and training speed was insufficient for meaningful results. Migrating to Google Colab resolved the GPU problem but introduced session timeout issues: Colab's free tier disconnects after approximately 90 minutes of inactivity and limits sessions to 3 hours 50 minutes. The anti-disconnection JavaScript script and the `CheckpointCallback` were engineering responses to these constraints. A second challenge was the pickling error caused by the custom reward wrapper: Stable-Baselines3's multiprocessing requires all objects to

be serialisable with cloudpickle, and our `SoccerRewardShapingWrapper` initially failed. We resolved this by implementing a custom `__reduce__` method.

What We Would Do Differently. If we were to start this project again, we would begin with a short integration test — a 1 000-step training run — before committing to a full training session. This would have identified the checkpoint and serialisation bugs much earlier. We would also set up TensorBoard monitoring from the very first run, which provides a real-time view of whether the agent is actually learning, rather than waiting for a full training session to conclude before inspecting the results. Finally, we would spend more time on reward coefficient sensitivity analysis before the main training run.

Impact on Future Development. This project has deepened our appreciation for the gap between reading about an algorithm and actually implementing it in a real system. The engineering challenges — serialisation, multiprocessing, cloud infrastructure, reward engineering — are as important as the mathematical theory, and they are rarely discussed in textbooks. We now feel equipped to approach future multi-agent RL projects with a more structured experimental methodology: start small, verify early, monitor continuously, and iterate. The field of multi-agent deep reinforcement learning remains an open research area with many unsolved problems, and this project has given us a foundation from which to explore it further.

7.1 Summary

This chapter provided a personal reflection on the project experience, covering the skills and knowledge developed, the decision-making approach adopted, the challenges encountered, and the lessons learned. It concludes the main body of this report. The References section follows, listing all sources cited throughout this work.

Note on References vs. Bibliography: The next section is "References," which will be automatically generated by BibTEX. A "References" list contains only the sources cited in the report, whereas a "Bibliography" also includes sources read but not cited. This report uses "References" only.

References

Bellman, R. (1957), *Dynamic Programming*, Princeton University Press, Princeton, NJ.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J. and Zaremba, W. (2016), 'OpenAI Gym'.

Farama Foundation (2022), 'Shimmy: Gymnasium and PettingZoo wrappers for existing reinforcement learning environments'.
URL: *https://github.com/Farama-Foundation/Shimmy*

Gupta, J. K., Egorov, M. and Kochenderfer, M. (2017), Cooperative multi-agent control using deep reinforcement learning, *in* 'International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS) – Adaptive Learning Agents Workshop'.

Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I. (2017), Multi-agent actor-critic for mixed cooperative-competitive environments, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 30.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015), 'Human-level control through deep reinforcement learning', *Nature* **518**(7540), 529–533.

Ng, A. Y., Harada, D. and Russell, S. J. (1999), Policy invariance under reward transformations: Theory and application to reward shaping, *in* 'International Conference on Machine Learning (ICML)', pp. 278–287.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N. (2021), 'Stable-Baselines3: Reliable reinforcement learning implementations', *Journal of Machine Learning Research* **22**(268), 1–8.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017), 'Proximal policy optimization algorithms', *arXiv preprint arXiv:1707.06347* .

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. et al. (2016), 'Mastering the game of Go with deep neural networks and tree search', *Nature* **529**, 484–489.

Sutton, R. S. and Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2nd edn, MIT Press, Cambridge, MA.

Sutton, R. S., McAllester, D., Singh, S. and Mansour, Y. (1999), Policy gradient methods for reinforcement learning with function approximation, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 12.

Tassa, Y., Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T. and Heess, N. (2020), 'dm_control: Software package for physics-based simulation and reinforcement learning'.
URL: *https://github.com/deepmind/dm_control*

Terry, J. K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L. S., Dieffendahl, C., Horsch, C., Perez-Vicente, R. et al. (2021), 'PettingZoo: Gym for multi-agent reinforcement learning', *Advances in Neural Information Processing Systems (NeurIPS)* **34**.

Terry, J. K., Black, B. and Jayakumar, M. (2021), 'SuperSuit: Simple microwrappers for RL environments'.
URL: *https://github.com/Farama-Foundation/SuperSuit*

Watkins, C. J. C. H. and Dayan, P. (1992), 'Q-learning', *Machine Learning* **8**(3–4), 279–292.

Williams, R. J. (1992), 'Simple statistical gradient-following algorithms for connectionist reinforcement learning', *Machine Learning* **8**(3–4), 229–256.

Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. and Wu, Y. (2022), 'The surprising effectiveness of PPO in cooperative multi-agent games', *Advances in Neural Information Processing Systems (NeurIPS)* **35**, 24611–24624.

Appendix A

Project Source Code Overview

This appendix provides a summary of the project's source code structure. The full source code is available in the project repository.

Repository Structure

```
projet_rl/
|-- src/
|   |-- env_setup.py           # MuJoCo environment + reward shaping
|   |-- train.py               # PPO training pipeline
|   |-- evaluate.py            # Model evaluation with rendering
|   |-- demos/
|   |   |-- q_learning_demo.py # Q-Learning on FrozenLake-v1
|   |   `-- dqn_demo.py        # DQN on CartPole-v1
|   `-- utils/
|       |-- analysis.py        # Statistical evaluation + graphs
|       `-- plot_training_curves.py  # TensorBoard log plotter
|-- models/                    # Saved model checkpoints
|-- results/                   # Generated graphs and figures
|-- Colab_Training_Notebook.ipynb  # Google Colab training notebook
|-- requirements.txt           # Python dependencies
`-- rapport/                   # This LaTeX report
```

Key Dependencies

Table A.1: Main Python dependencies used in this project.

Library	**Purpose**
`stable-baselines3`	PPO training framework
`pettingzoo`	Multi-agent environment API
`shimmy`	DmControl compatibility layer
`supersuit`	Environment vectorisation
`dm_control`	MuJoCo physics engine
`gymnasium`	FrozenLake, CartPole environments
`torch`	Deep learning (DQN network)
`tensorboard`	Training curve visualisation
`matplotlib`	Result plotting